JSON Refiner – Advanced Edition

Professional JSON Processing Tool with Gradio Interface

STUDENT NAME:

1. S.YOGAVARSHIKA
2. J.ABINA
3. B.AKSHANA
4. S.LATHIKA
5. P.PAVITHRA

REGISTER NUMBER:

1. 23126ER017
2. 23126ER001
3. 23126ER002
4. 23126ER009
5. 23126ER013

INSTITUTION : **SRI ADI CHUNCHANAGIRI WOMEN'S COLLEGE, CUMBUM**

DEPARTMENT : **B.Sc COMPUTER SCIENCE**

DATE OF SUBMISSION : **08/03/2026**

GITHUB REPOSITORY LINK : yogavarshikascs2026-collab/NM_JSON-Refiner-Advanced-Edition: JSON Refiner – Advanced Edition (github.com)

JSON Refiner – Advanced Edition

Professional JSON Processing Tool with Gradio Interface

Project Description

JSON Refiner – Advanced Edition is a professional JSON processing tool developed using Python with an interactive interface built using Gradio. The system is designed to simplify the process of working with JSON data by providing features such as parsing, validation, transformation, and refinement in a user-friendly environment. JSON is widely used for data exchange in modern applications, but handling unstructured or inconsistent JSON data can be challenging. This project helps solve that problem by providing an efficient platform that improves JSON data quality and formatting.

The system accepts unstructured key-value input from the user and converts it into a well-structured JSON format. Using Python's data handling capabilities, the tool automatically identifies and assigns appropriate data types such as strings, numbers, boolean values, null values, arrays, and objects. This automatic type detection helps ensure that the generated JSON data is accurate and consistent. The platform also supports different naming convention conversions including snake_case, camelCase, kebab-case, and PascalCase, allowing developers to standardize the format of their JSON keys according to their project requirements.

In addition to data formatting, the system also provides JSON transformation features such as flattening and unflattening. Flattening converts nested JSON structures into flat key-value pairs, which makes the data easier to process or analyze. Unflattening performs the reverse operation by reconstructing the original nested JSON structure from flattened data. These transformation capabilities help users manage complex JSON structures more efficiently.

The application also includes JSON schema validation to ensure the correctness of the generated JSON data. By comparing the JSON data against a defined schema, the system can identify structural errors and notify the user if the data does not follow the required format. This validation process improves data reliability and helps maintain data integrity.

The entire platform is implemented using Python and presented through a Gradio-based graphical user interface, which allows users to interact with the system easily through a web interface. With its combination of automatic data processing, transformation features, validation support, and interactive design, **JSON Refiner – Advanced Edition** provides a convenient and efficient solution for refining and managing JSON data while ensuring clean and production-ready output.

Scenarios

Scenario 1: Converting Key–Value Data into JSON

A user may have simple key–value data written in plain text format. Manually converting such data into proper JSON format can be time-consuming and error-prone. The system accepts input such as:

```
name: John
age: 28
active: true
```

The application automatically detects the correct data types such as string, integer, boolean, or float using the type inference mechanism. It then converts the input into a properly structured JSON format. This feature allows users to quickly generate clean and valid JSON without manually creating the entire structure.

Scenario 2: Standardizing JSON Key Naming Conventions

In many real-world applications, JSON data may contain inconsistent key naming styles such as **FirstName**, **first_name**, or **firstName**. Such inconsistencies can reduce readability and make data processing difficult.

The system provides a case conversion module that allows users to convert JSON keys into a consistent naming style such as:

- snake_case
- camelCase
- kebab-case
- PascalCase

This ensures that JSON data follows a uniform naming convention, making it easier to maintain and integrate with other systems.

Scenario 3: JSON Data Validation Using Schema

Users may need to verify whether their JSON data follows a predefined structure. The system provides a JSON validation feature that checks the JSON data against a given schema.

The validation module ensures that:

- Required fields are present
- Data types match the expected format
- The JSON structure is valid

If validation fails, the system displays clear error messages indicating the problem. This helps users correct mistakes and ensures the JSON data is accurate and reliable.

Scenario 4: Flattening Nested JSON Structures

In some cases, JSON data may contain deeply nested objects, which can make analysis or processing difficult. The system provides a flattening feature that converts nested JSON structures into simple dot-notation key-value pairs.

For example:

Original JSON

```
{
 "user": {
   "name": "John",
   "profile": {
     "age": 28
   }
 }
}
```

Flattened JSON

```
{
 "user.name": "John",
 "user.profile.age": 28
}
```

Flattening simplifies data processing and makes JSON easier to analyze or display in tabular formats.

Scenario 5: Restoring JSON Structure (Unflattening)

After flattening JSON data for easier processing, users may want to restore the original hierarchical structure. The system includes an unflattening feature that reconstructs nested JSON objects from flattened dot-notation keys.

For example:

Flattened JSON

```
{
 "user.name": "John",
 "user.profile.age": 28
}
```

Restored JSON

```
{
 "user": {
   "name": "John",
   "profile": {
     "age": 28
   }
 }
}
```

This functionality allows users to easily switch between flattened and nested formats while preserving the original data structure.

Architecture Overview

The **JSON Refiner Advanced** system is designed as a modular application that processes and refines JSON data through multiple transformation and validation stages. The architecture consists of three main layers: the **User Interface Layer**, the **Processing Layer**, and the **Data Handling Layer**. The User Interface Layer is built using the Gradio framework, which allows users to interact with the system through a web-based interface where they can input data, select transformation options, and view results. The Processing Layer contains the core logic functions responsible for performing operations such as key–value conversion, case style transformation, JSON validation, merging, flattening, and unflattening. These functions analyze the input data, apply the required transformations, and generate refined JSON output. The Data Handling Layer manages JSON parsing, schema validation, and history storage using Python libraries. This layered architecture ensures that the system is organized, scalable, and easy to maintain while providing efficient JSON processing capabilities.

Core Technologies

- Python – Main programming language used to implement the JSON processing logic.
- Gradio Framework – Used to create an interactive and user-friendly web interface.
- JSON Module – Used to parse, generate, and format JSON data.
- JSONSchema Library – Used to validate JSON data based on schema rules.
- Regular Expressions (Regex) – Used for key conversion and string pattern matching.
- Google Colab – Cloud-based environment used for writing and running the project code.
- HTML Rendering (via Gradio) – Used to display the user interface in the browser.
- Python Functions – Used to implement different processing features like flattening and merging.
- Enum and Data Structures – Used for managing case styles and structured data processing.

Knowledge Prerequisites

- Basic Python Programming – Understanding functions, loops, and conditional statements.
- Data Structures in Python – Knowledge of lists, dictionaries, and tuples.
- JSON Data Format – Understanding JSON objects, arrays, keys, and values.
- JSON Data Types – Knowledge of string, integer, float, boolean, and null types.
- Regular Expressions (Regex) – Understanding basic pattern matching techniques.
- Gradio Framework Basics – Understanding UI components like textbox, buttons, and outputs.
- JSON Schema Validation – Basic idea about validating data using schema rules.
- Google Colab Environment – Ability to run code cells and install libraries.
- Error Handling Concepts – Understanding try–except blocks in Python.

Software Requirements

- Python 3.8 or higher – Required for running the JSON processing program.
- Google Colab – Used as the main development and execution platform.
- Gradio Library – Used to create the graphical user interface.
- JSON Library – Built-in Python library used for JSON manipulation.
- JSONSchema Library – Used for validating JSON data against schemas.
- Regex Library (re module) – Used for string pattern matching and case conversion.
- pip Package Manager – Used to install required Python packages.
- Internet Connection – Required to access Google Colab and install libraries.
- Web Browser – Chrome, Firefox, Edge, or Safari to run the Gradio interface.

Hardware Requirements

- Processor – Intel i3 or any modern processor capable of running a web browser.
- RAM – Minimum 2 GB RAM recommended for smooth performance.
- Storage – At least 500 MB free storage to save project files and outputs.
- Display Resolution – Minimum 1024 × 768 resolution for clear interface viewing.
- Keyboard and Mouse – Required for interacting with the system.
- Internet Connectivity – Required to access Google Colab and share the Gradio interface.
- Laptop or Desktop Computer – Recommended for better usability compared to mobile devices.

Project Workflow

Phase 1: Environment Setup & Core Architecture

Activity 1.1: Install Dependencies & Configure Environment

- Set up the development environment using **Google Colab**.
- Install required Python libraries such as **gradio** and **jsonschema**.
- Import necessary modules including **gradio, json, re, and jsonschema**.
- Verify that all libraries are successfully installed and working.
- Prepare the notebook environment for executing the JSON Refiner application.

Activity 1.2: Design Core Processing Functions

- Create the **infer_type()** function for automatic data type detection.
- Implement **convert_case()** function to convert key naming styles.
- Define a **history storage list** to keep track of previous outputs.
- Implement error handling using **try–except blocks**.
- Organize all processing functions to handle JSON transformation tasks.

Phase 2: Core JSON Processing Engine

Activity 2.1: Implement Type Inference System

- Develop the **infer_type()** function for detecting correct data types.
- Detect boolean values such as **true, false, yes, no, on, off**.
- Handle **null values** such as null, none, or nil.
- Convert numeric values into **integers or floating point numbers**.
- Treat unrecognized values as **strings by default**.
- Ensure accurate type conversion when creating JSON output.

Activity 2.2: Build Key Normalization & Case Conversion

- Implement **snake_case conversion** for JSON keys.
- Implement **camelCase conversion** for JSON keys.
- Implement **PascalCase conversion** for JSON keys.
- Implement **kebab-case conversion** for JSON keys.
- Apply selected case style using the **convert_case()** function.
- Ensure consistent key naming across the generated JSON structure.

Phase 3: Validation & Transformation Features

Activity 3.1: Implement JSON Schema Validation

- Create **validate_json()** function for validating JSON data.
- Use the **jsonschema validate() method** for schema validation.
- Parse the JSON input before validation.
- Display **validation success or error messages** to the user.
- Ensure the JSON structure matches the given schema definition.

Activity 3.2: Develop JSON Transformation Methods

- Implement **merge_json()** function to combine two JSON objects.
- Implement **flatten_json()** to convert nested JSON into flat structure.
- Implement **unflatten_json()** to reconstruct nested JSON structure.
- Implement **remove null values option** during core refining.
- Maintain transformation results inside the **history store**.
- Test transformation functions using different JSON inputs.

Phase 4: Gradio Interface Development

Activity 4.1: Design Core Refining Interface

- Create the **Gradio Blocks interface** for the application.
- Add input textbox for **JSON or key-value data input**.
- Add an additional textbox for **merging another JSON object**.
- Provide a dropdown for selecting **case style conversion**.
- Add a checkbox option for **removing null values**.
- Display processed JSON in a **formatted output textbox**.
- Include status messages to indicate system results.

Activity 4.2: Add Interactive Control Buttons

- Create **Core Refining button** to process key-value input into JSON.
- Create **Validation button** to validate JSON against schema.
- Create **Case Conversion button** to convert key naming styles.
- Create **Merge JSON button** to combine two JSON objects.
- Create **Flatten button** to simplify nested JSON structures.
- Create **Unflatten button** to reconstruct nested JSON structures.
- Create **History button** to display previous outputs.
- Create **Guide button** to show system usage instructions.
- Create **Clear button** to reset inputs and outputs.

Phase 5: Integration, Testing & Deployment

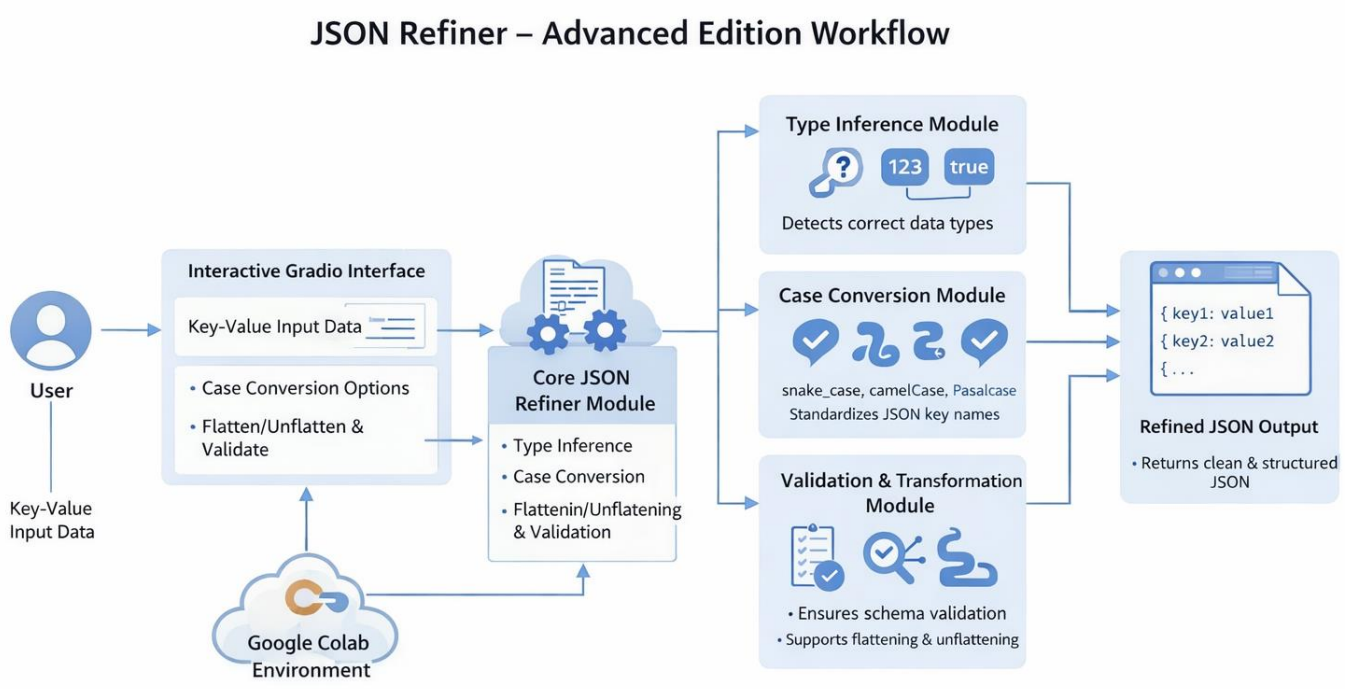
Activity 5.1: Google Colab Deployment

- Integrate all processing functions with the **Gradio interface**.
- Connect input components with their corresponding backend functions.
- Ensure proper **data flow between UI and processing logic**.
- Launch the interface using **demo.launch()** in Google Colab.
- Verify that the interface loads correctly in the browser.

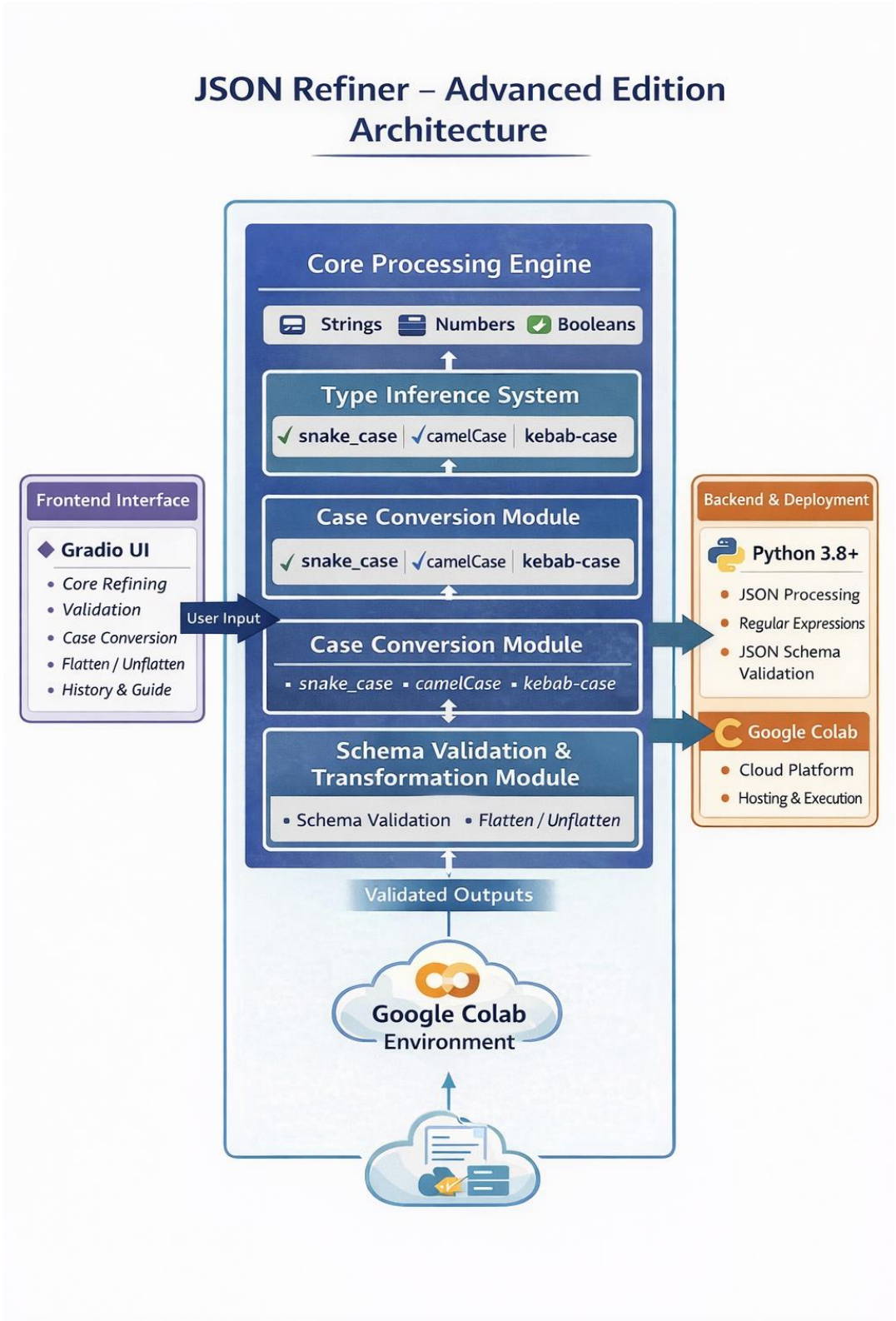
Activity 5.2: System Testing

- Test **key-value input conversion into JSON format**.
- Verify **automatic type detection** for different data types.
- Test **case conversion styles** for JSON keys.
- Validate JSON data using different schema definitions.
- Test merging of multiple JSON objects.
- Test flattening and unflattening of nested JSON structures.
- Verify history storage and output display functionality.

Technical Architecture



Project Structure



Milestone 1: Environment Setup and Core System Initialization

Activity 1.1: Installing Required Libraries

In this step, the required Python libraries needed for the JSON Refiner system are installed. The project is developed using the cloud-based environment Google Colab, which allows users to run Python programs directly from a web browser without installing software locally. The main libraries used in the project include the Gradio library for building an interactive user interface, the JSON module for handling JSON data structures, the Regular Expression module for text processing, and the JSONSchema library for validating JSON structures. These libraries provide the foundation for building the JSON transformation and validation system.

```
[ ] !pip install gradio jsonschema

Requirement already satisfied: gradio in /usr/local/lib/python3.12/dist-packages (5.50.0)
Requirement already satisfied: jsonschema in /usr/local/lib/python3.12/dist-packages (4.26.0)
```

```
[ ] import gradio as gr
    import json
    import re
    from jsonschema import validate

    # -------- CORE LOGIC --------
```

Activity 1.2: Core System Initialization

After installing the required libraries, the core system components of the JSON Refiner are initialized. A global list called *history_store* is created to store the history of processed JSON outputs. This enables the system to maintain a record of previous transformations performed by the user. Maintaining transformation history improves usability and allows users to review previously generated outputs within the application.

```
[ ] history_store = []
```

```
# -------- HISTORY --------

def show_history():

    if not history_store:
        return "No history","🕒 Empty"

    return "\n\n".join(history_store),"🕒 History loaded"
```

Milestone 2: JSON Processing and Type Inference

Activity 2.1: Automatic Data Type Detection

The JSON Refiner system implements an automatic type detection mechanism through the *infer_type()* function. This function analyzes input values and determines the appropriate data type such as boolean, null, integer, float, or string. For example, values like *true*, *false*, and *null* are automatically converted into their corresponding JSON types instead of remaining as strings. This improves the accuracy of generated JSON structures and ensures compatibility with JSON validation processes.

```python
history_store = []

def infer_type(v):
    v = str(v).strip()
    if v.lower() in ["true","yes","on"]: return True
    if v.lower() in ["false","no","off"]: return False
    if v.lower() in ["null","none","nil"]: return None
    try:
        if "." in v: return float(v)
        return int(v)
    except:
        return v
```

Activity 2.2: JSON Key Case Conversion

Different applications use different naming conventions for JSON keys. To handle this, the JSON Refiner includes a *convert_case()* function that converts keys into different naming styles such as snake_case, camelCase, PascalCase, and kebab-case. This is implemented using regular expressions to detect uppercase letters and separators. The feature helps developers maintain consistent naming conventions in JSON data structures.

```python
def convert_case(key, style):

        if style=="snake_case":
            temp=re.sub(r'[\s\-]+','_',key)
            snake=re.sub(r'(?<!^)(?=[A-Z])','_',temp).lower()
            return re.sub(r'_{2,}','_',snake)

        elif style=="camelCase":
            parts=re.split(r'[_\-\s]+',key)
            return parts[0].lower()+''.join(p.capitalize() for p in parts[1:])

        elif style=="PascalCase":
            return ''.join(p.capitalize() for p in re.split(r'[_\-\s]+',key))

        elif style=="kebab-case":
            temp=re.sub(r'[\s_]+','-',key)
            kebab=re.sub(r'(?<!^)(?=[A-Z])','-',temp).lower()
            return re.sub(r'-{2,}','-',kebab)

        return key
```

Milestone 3: JSON Refining and Transformation

Activity 3.1: Core JSON Refining

The core refining module is responsible for converting raw input data into structured JSON format. The *core_refine()* function accepts key-value input text and processes it to generate a valid JSON object. The function first checks whether the input is already in JSON format; if not, it converts the key-value list into a JSON structure. Additional features include case conversion for keys and optional removal of null values. The processed output is then formatted using JSON indentation for readability and stored in the system history.

```
def core_refine(text,style,remove_nulls):

    if not text.strip():
        return "", "⚠ Enter data"

    try:
        try:
            result=json.loads(text)
            status="✅ JSON detected"
        except:
            result={}
```

Milestone 4: Validation and JSON Operations

Activity 4.1: JSON Schema Validation

To ensure data correctness, the JSON Refiner system supports schema validation using the JSONSchema library. The *validate_json()* function verifies whether the generated JSON data follows a given schema definition. If the JSON structure matches the schema, the system displays a successful validation message. Otherwise, an error message is returned describing the validation issue. This feature helps maintain structured and reliable JSON data.

```
def validate_json(json_str,schema_str):

    try:
        data=json.loads(json_str)

        if schema_str.strip()==="":
            return json_str,"✅ JSON Valid"

        validate(instance=data,schema=json.loads(schema_str))

        return json_str,"✅ Schema Valid"

    except Exception as e:
        return json_str,f"❌ Validation Error {str(e)}"
```

Activity 4.2: JSON Merging

The JSON Refiner system includes a merging module that allows two JSON objects to be combined into a single structure. The *merge_json()* function takes two JSON inputs and merges their key-value pairs. If both objects contain the same key, the second JSON value overrides the first. This feature simplifies data integration from multiple sources.

```python
def merge_json(json1,json2):

    try:
        d1=json.loads(json1)
        d2=json.loads(json2)

        merged={**d1,**d2}

        out=json.dumps(merged,indent=4)

        history_store.append(out)

        return out,"🔗 JSON Merged"

    except:
        return json1,"⚠ Invalid JSON"
```

Milestone 5: JSON Transformation Features

Activity 5.1: Flatten JSON

Flattening is a process used to convert nested JSON objects into a single-level structure. The *flatten_json()* function traverses nested dictionaries and converts hierarchical keys into dot notation format. This transformation simplifies data storage and makes JSON structures easier to analyze and manipulate.

```python
def flatten_json(json_str):

    try:
        data=json.loads(json_str)
        flat={}

        def walk(x,name=""):
            if isinstance(x,dict):
                for a in x:
                    walk(x[a],f"{name}{a}.")
            else:
                flat[name[:-1]]=x

        walk(data)

        out=json.dumps(flat,indent=4)

        history_store.append(out)

        return out,"📉 Flattened"

    except:
        return json_str,"⚠ Invalid JSON"
```

Activity 5.2: Unflatten JSON

The unflatten operation reverses the flattening process by reconstructing nested JSON structures from dot-separated keys. The *unflatten_json()* function splits keys using the dot separator and rebuilds the hierarchical structure accordingly. This allows users to restore original nested data from flattened JSON representations.

```
def unflatten_json(json_str):

    try:
        data=json.loads(json_str)

        res={}

        for k,v in data.items():
            parts=k.split(".")
            d=res

            for p in parts[:-1]:
                d=d.setdefault(p,{})

            d[parts[-1]]=v

        out=json.dumps(res,indent=4)

        history_store.append(out)

        return out,"🔄 Unflattened"

    except:
        return json_str,"⚠ Invalid JSON"
```

Milestone 6: User Interface Development

Activity 6.1: Interactive Interface Design

To make the system user-friendly, an interactive graphical interface is developed using the Gradio framework. The interface provides input text areas for JSON data, schema definitions, and additional JSON objects for merging. Dropdown menus and checkboxes allow users to select case conversion styles and remove null values. The processed output is displayed in a formatted output panel along with system status messages.

```
with gr.Blocks(title="JSON Refiner Pro") as demo:

    gr.Markdown("# JSON Refiner Advanced")

    with gr.Row():

        with gr.Column():

            inp=gr.Textbox(label="Input JSON / List",lines=10)

            inp2=gr.Textbox(label="Merge JSON (optional)",lines=5)

            schema=gr.Textbox(label="Schema",lines=3)

            style=gr.Dropdown(["snake_case","camelCase","PascalCase","kebab-case"],value="snake_case")

            rem=gr.Checkbox(label="Remove Nulls",value=True)

        with gr.Column():

            status=gr.Markdown("Status: Ready")

            # FIXED OUTPUT BOX
            out=gr.Textbox(label="Output",lines=18,interactive=False,show_copy_button=True)
```

Milestone 7: Utility Features

Activity 7.1: Guide Feature

The JSON Refiner system includes a built-in **Guide feature** that helps users understand how to use the application. When the guide button is clicked, the system displays instructions describing the main functionalities such as core refining, validation, case conversion, merging JSON, flattening, unflattening, and viewing history. This feature improves usability by providing quick instructions directly within the application interface.

```
def show_guide():

    text="""
Guide:

Core Refining → Convert Key:Value to JSON
Validation → Validate JSON with schema
Case Conversion → Change naming style
Merge JSON → Merge two JSON objects
Flatten → Convert nested JSON
Unflatten → Restore structure
History → Show previous results
"""

    return text,"📜 Guide"
```

Activity 7.2: Clear Feature

The Clear feature allows users to quickly reset the system interface. When the clear button is pressed, all input fields and output results are cleared, and the system status is reset to the default state. This functionality improves the user experience by allowing users to start new JSON processing tasks without manually deleting previous inputs.

```
def clear_all():
    return "", "", "Status: Ready"
```

Milestone 8: Integration, Testing and Deployment

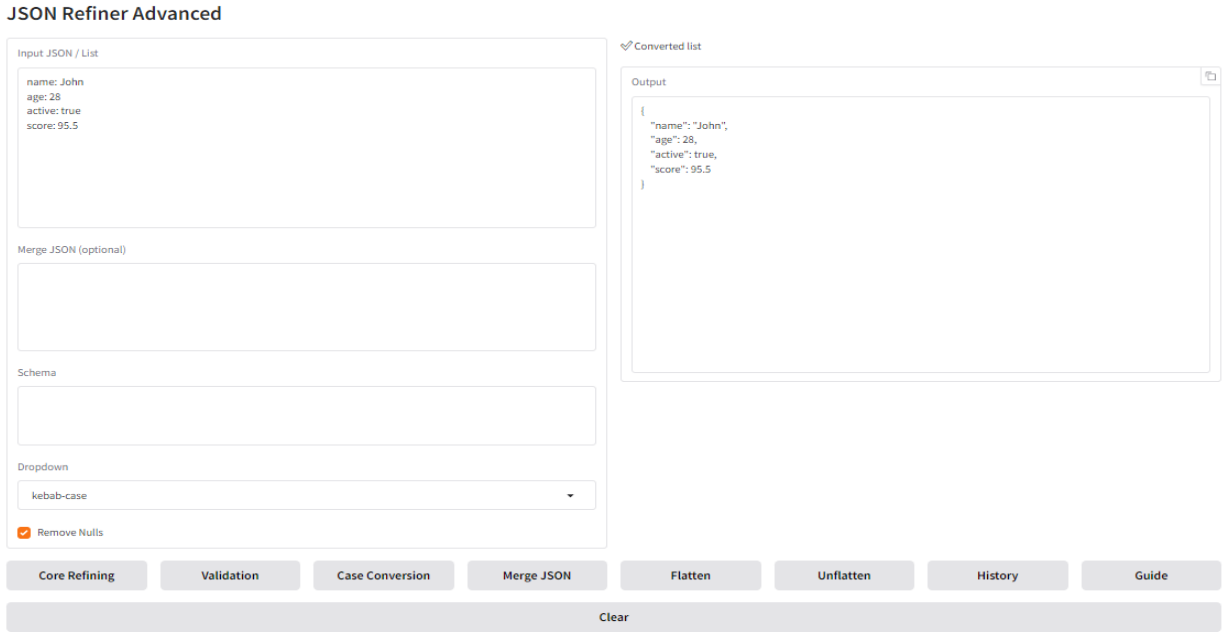
Activity 8.1: System Deployment

After integrating all processing modules and user interface components, the complete system is executed using the **Gradio launch function**. The demo.launch() command starts the application and generates a web-based interface that allows users to interact with the JSON Refiner system through a browser. This step completes the system integration and enables real-time JSON processing operations.

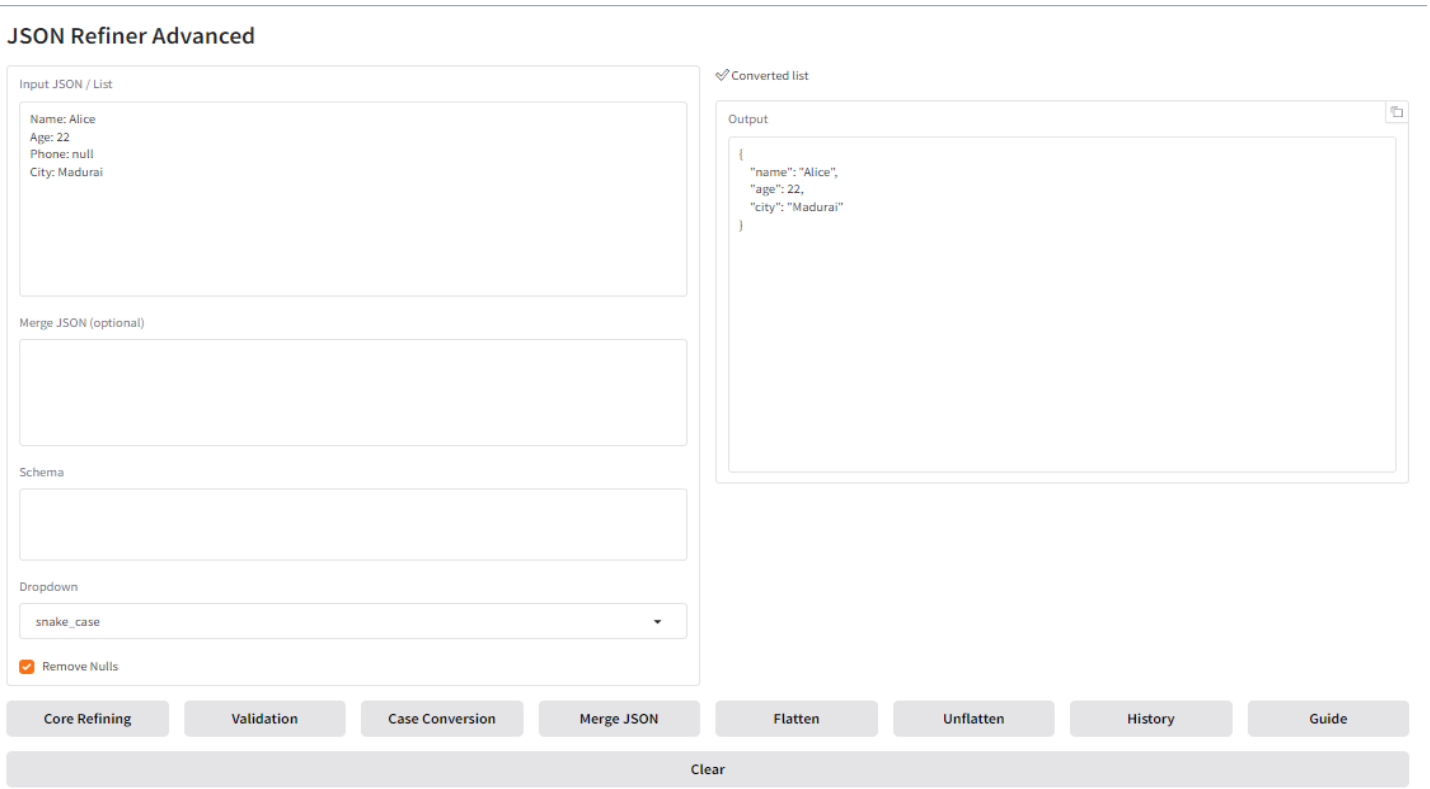
```
demo.launch()
```

A live interactive version of the **JSON Refiner Advanced Edition** is deployed using Gradio and can be accessed through the following link: https://c5f9cc995544a8c479.gradio.live/

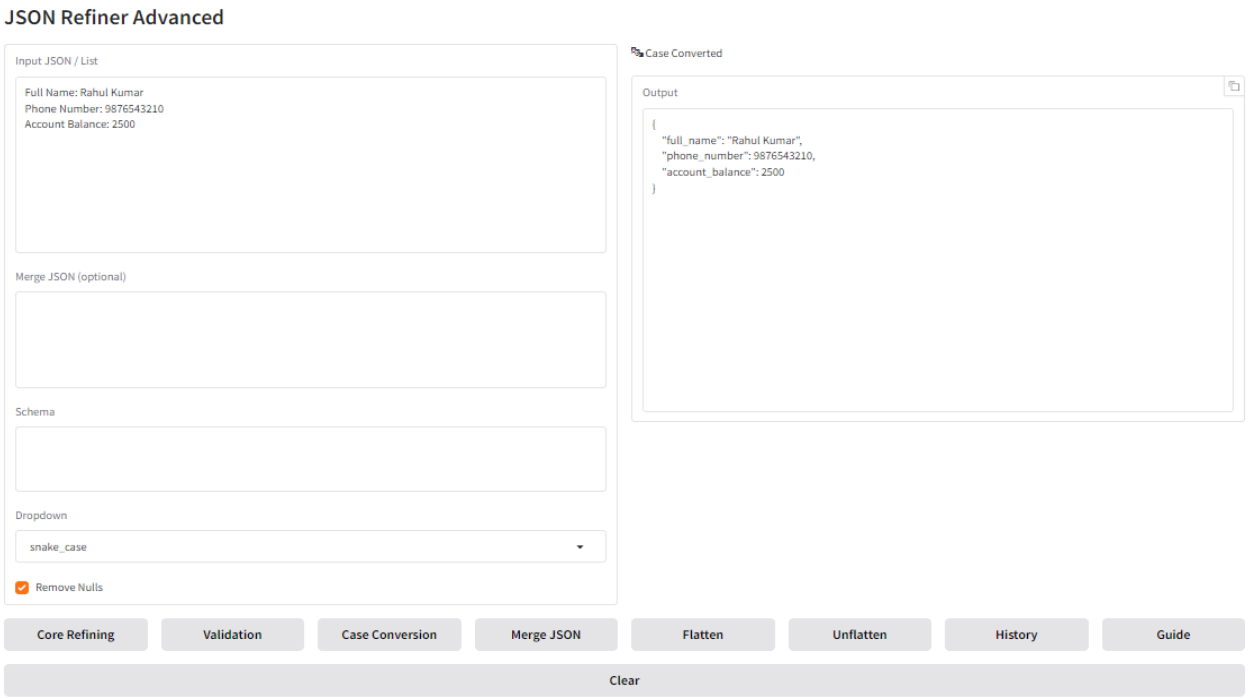
Activity 8.2: Function Testing



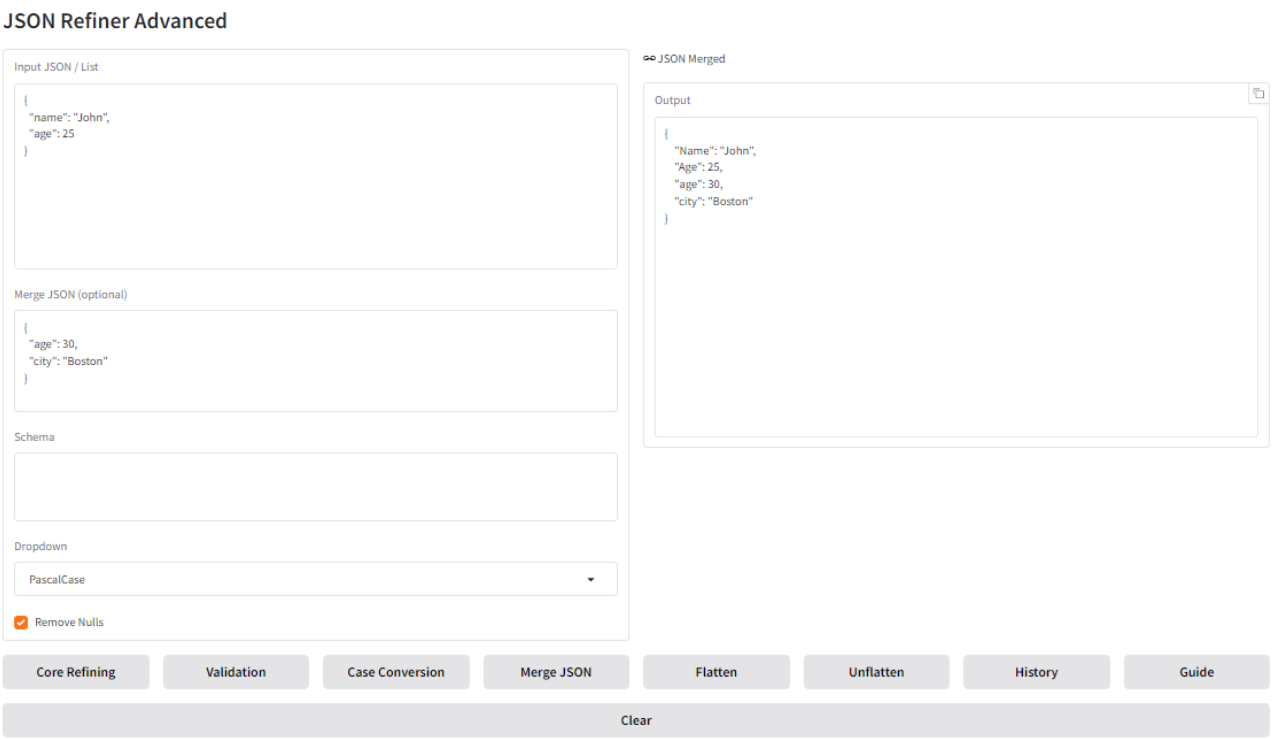
CODE REFINING



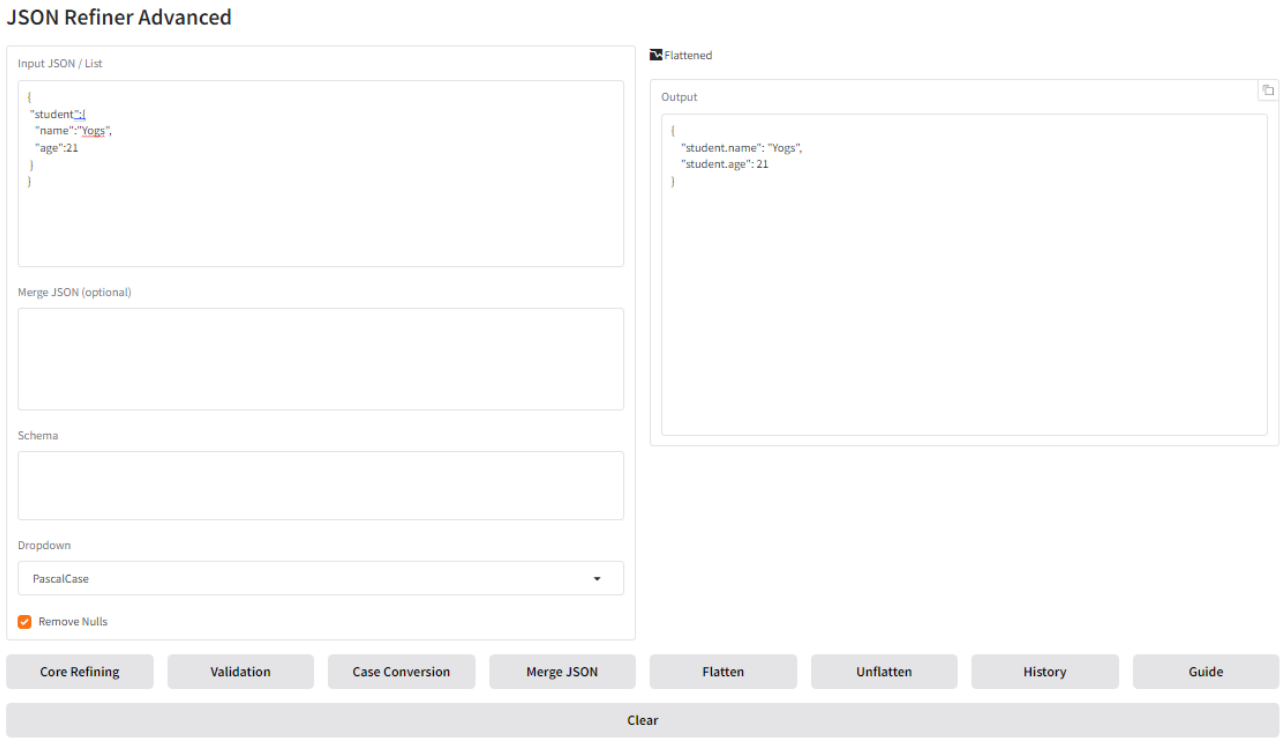
Null Removal Test



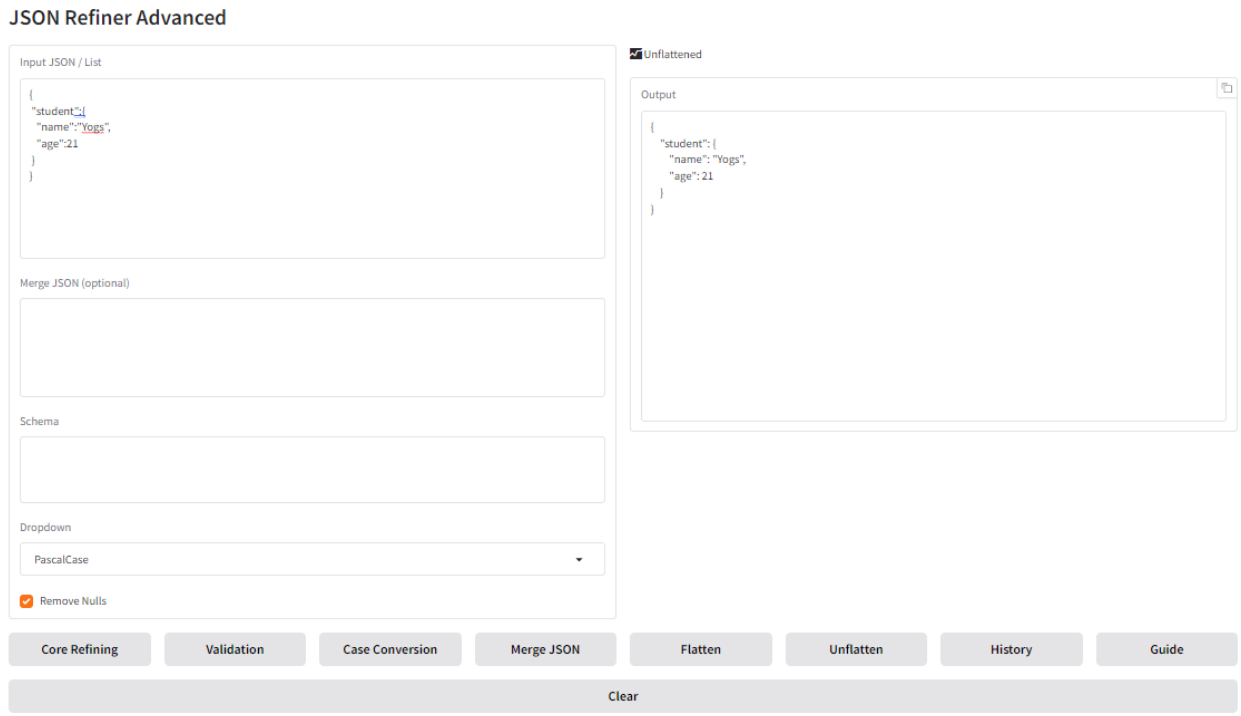
CASE CONVERSION



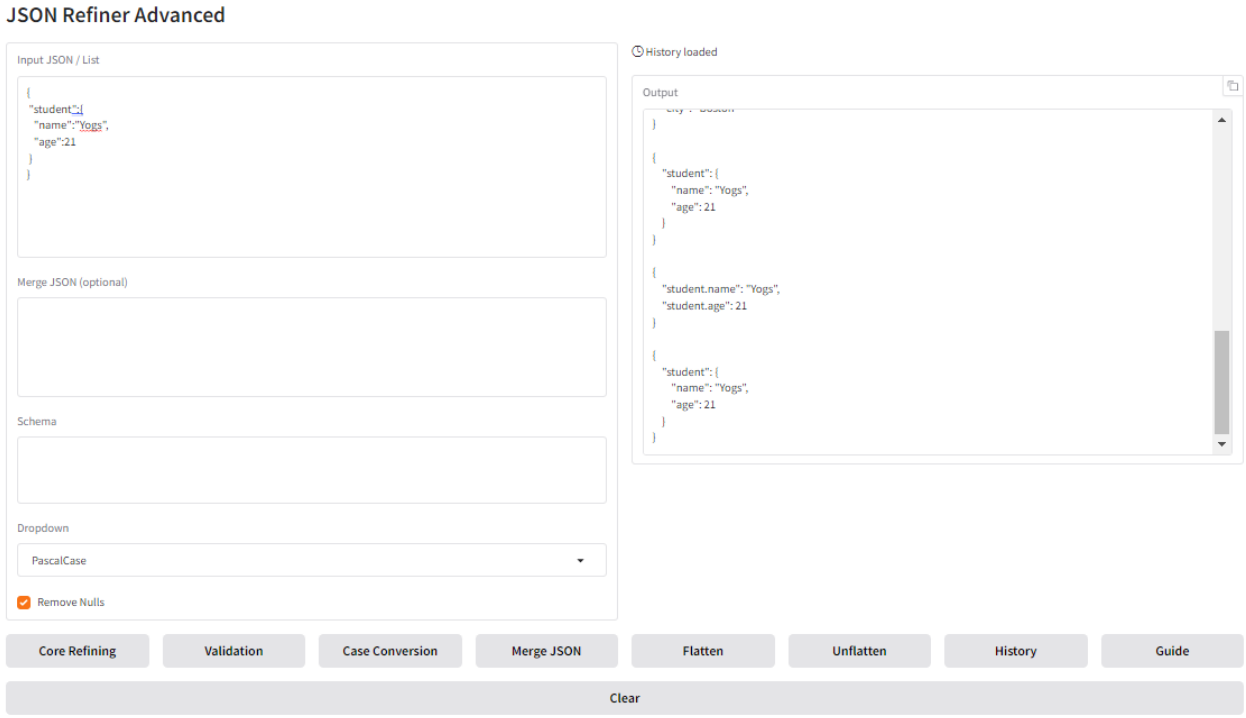
MERGE JASON



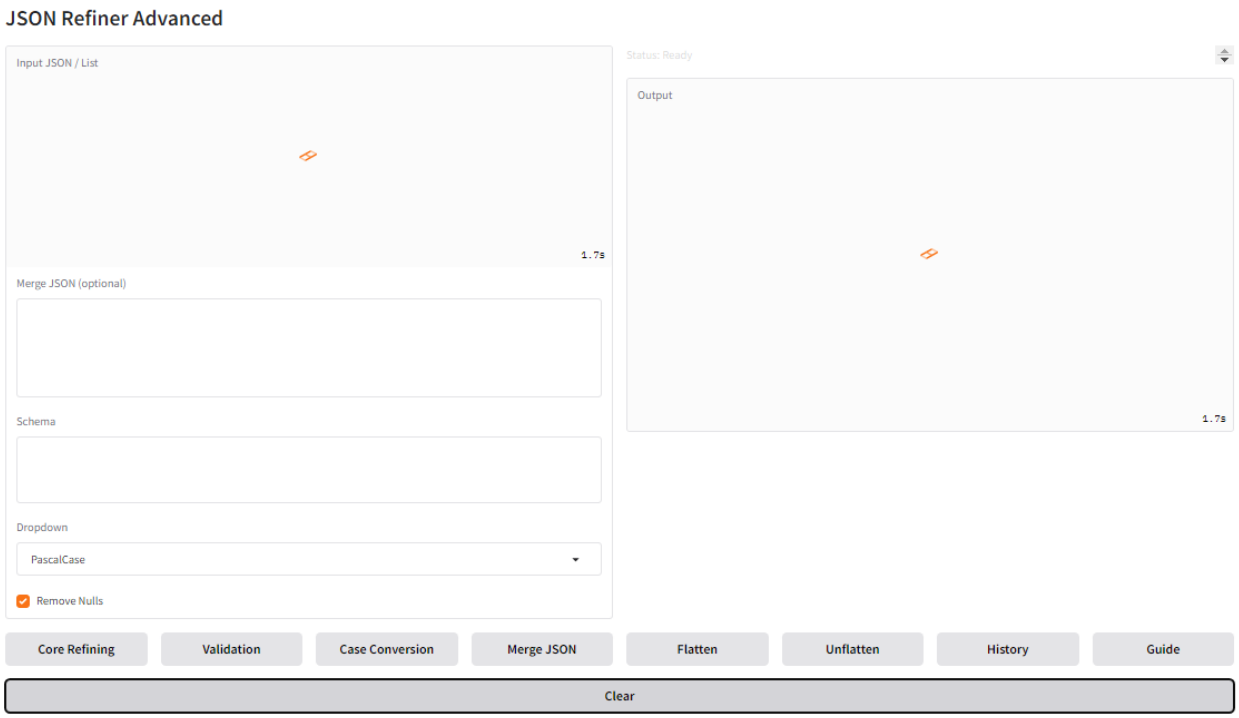
FLATTEN



UNFLATTEN



HISTROY


JSON Refiner Advanced
Input JSON / List
1.7s
Status: Ready
Output
1.7s
Merge JSON (optional)
Schema
Dropdown
PascalCase
Remove Nulls
Core Refining
Validation
Case Conversion
Merge JSON
Flatten
Unflatten
History
Guide
Clear

CLEARING